

# LangChain in Production

## The Hard Parts Everyone Skips

AI with Surya · Enterprise AI Engineering Blueprint · EAI-001

**Purpose:** A practical reference for moving LangChain applications beyond prototypes into secure, observable, evaluable, reliable enterprise systems.

### 1. Production Mental Model

A production AI application is not simply User → LLM → Answer. LangChain should sit inside a broader application architecture that handles identity, policy, data access, reliability and operations.

| Layer            | Primary responsibility                                                      |
|------------------|-----------------------------------------------------------------------------|
| Experience       | Web, API, Teams/Slack or other user interfaces                              |
| Edge             | API gateway, authentication, authorization, rate limiting                   |
| Application      | Business rules, request validation, orchestration boundary                  |
| AI orchestration | LangChain / LangGraph, prompts, tools, retrievers, structured output        |
| Knowledge        | Document stores, vector DB, metadata, enterprise search                     |
| Models           | Azure OpenAI / OpenAI / other approved model providers                      |
| Controls         | PII protection, prompt-injection defenses, tool policies, output validation |
| Operations       | Tracing, evaluation, cost, latency, errors, alerts, auditability            |

### 2. Core LangChain Building Blocks

#### Models

Keep provider-specific configuration behind a clean model boundary so model selection and migration are easier.

#### Prompts

Treat prompts as versioned application assets. Review, test and evaluate them like code.

#### Retrievers

Use top-K, metadata filtering, hybrid search and, where appropriate, re-ranking.

#### Tools

Expose only approved enterprise capabilities and treat every tool call as a security and reliability boundary.

#### Structured output

Prefer typed, validated outputs when downstream code needs predictable fields.

#### State and orchestration

Use explicit workflow/state management for branching, retries, human approval and multi-step agent behavior.

### 3. Enterprise RAG Flow

Question → Query transformation → Retrieval → Filtering/Re-ranking → Context construction → Model → Validation → Response

Production retrieval must consider document freshness, metadata quality, authorization, chunk quality, embedding quality and evaluation.

### 4. Secure Vector Store Pattern

**Principle:** credentials and environment-specific settings belong outside source code.

```
from langchain_openai import AzureOpenAIEmbeddings from langchain_milvus import Milvus import os
embeddings = AzureOpenAIEmbeddings( azure_deployment=os.getenv("AZURE_OPENAI_EMBEDDING_DEPLOYMENT"),
api_version=os.getenv("AZURE_OPENAI_API_VERSION"), ) vector_db = Milvus(
embedding_function=embeddings, connection_args={ "uri": os.getenv("MILVUS_URI"), "token":
os.getenv("MILVUS_TOKEN"), }, collection_name="enterprise_knowledge_base", )
```

### 5. LangChain vs. LangGraph

| Need                          | Typical fit                          |
|-------------------------------|--------------------------------------|
| Prompt + model + retrieval    | LangChain components                 |
| Tool calling                  | LangChain tools / model integrations |
| Stateful, branching workflows | LangGraph                            |
| Human approval checkpoints    | LangGraph                            |
| Multi-step agent loops        | LangGraph                            |

### 6. Guardrails & Security

Do not rely on the LLM to enforce enterprise security. Authentication and authorization should happen outside the model, while input/output controls, prompt-injection defenses, PII handling and tool policies should be explicit.

Authentication · Authorization · PII protection · Prompt-injection controls · Tool allowlists · Tenant isolation · Secret management · Audit logging · Output validation

### 7. Observability

Trace the complete request path: user request, prompt, retrieval, retrieved documents, tool calls, model invocation and final response.

| Metric                     | Why it matters          |
|----------------------------|-------------------------|
| Latency                    | User experience and SLA |
| Token usage / cost         | Budget control          |
| Retrieval relevance        | Knowledge quality       |
| Model errors               | Reliability             |
| Tool failures              | Agent reliability       |
| Groundedness / correctness | Answer quality          |

## 8. Evaluation

A handful of manual questions is not an evaluation strategy. Build a representative dataset and measure retrieval and answer quality before and after changes.

- Retrieval relevance
- Context precision / recall where measurable
- Answer correctness
- Groundedness
- Response relevance
- Latency
- Cost per request
- Regression after model/prompt/index changes

## 9. Cost & Performance

At enterprise scale, model and retrieval costs become architecture concerns. Use caching, sensible model routing, token limits, context optimization, batching where appropriate, and retrieval improvements before simply increasing model size.

## 10. Reliability & Failure Handling

Design for model timeouts, provider errors, vector-store failures and tool failures. Use bounded retries with backoff, timeouts, fallbacks where justified, circuit-breaking patterns where appropriate, and graceful user-facing errors.

## 11. Production Readiness Checklist

- Secrets managed securely
- Authentication and authorization
- Input validation
- Prompt-injection controls
- PII protection
- Retrieval evaluation
- LLM evaluation
- Tracing and observability
- Cost monitoring
- Rate limiting
- Timeout and retry strategy
- Model fallback strategy where justified
- Prompt/version management
- Dependency pinning and upgrade process
- Automated tests

■ CI/CD

■ Production runbook

## 12. Enterprise AI Engineer Perspective

**Beginner:** How do I use LangChain?

**AI Engineer:** Where should LangChain sit in my architecture?

**Senior AI Engineer:** Do I need LangChain for this component?

**Enterprise Architect:** How will this behave under scale, security, audit, failure and cost constraints?

## 13. Key Takeaways

- LangChain is an orchestration/component layer, not the entire enterprise architecture.
- Production quality depends heavily on security, retrieval, evaluation, observability and reliability.
- Keep provider credentials and environment configuration outside application source code.
- Use explicit stateful orchestration when workflows become complex or agentic.
- Evaluate continuously when changing models, prompts, embeddings, chunking or retrieval.

AI with Surya · Enterprise AI Engineering Series